

Chennai District Intelligence: Tech Stack

The technology behind every function, split into what is built now and what is planned. Checked against each project's actual imports and package files.

1. Collecting real data (built, unchanged)

Function	Tech stack
News collection: Google News (English and Tamil), The Hindu, TOI, DT Next, NIE and Dinamani feeds, Dinamalar sitemap	Python 3.11, requests with truststore, feedparser, lxml, trafilatura (article text, only where robots.txt allows), langdetect, YAML config, optional PyMySQL
News relevance, department and complaint tagging	Regex place-name list for English and Tamil (handles Tamil word endings), keyword rules; optional OpenAI gpt-4o-mini (off)
IMD weather: observations, forecasts, warnings	Python, requests (IMD's JSON and HTML pages), BeautifulSoup, pandas, robots.txt checks, raw page snapshots
CPCB air quality	Python, requests, Playwright (headless browser reading the public map; CAPTCHA pages are not bypassed), pandas
CFM-DSS flood monitor: gates, gauges, bulletins	Python, requests, Playwright, pandas

2. Departmental sources (built, unchanged)

Function	Tech stack
Grievance portal: registration, login, filing, tracking	Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, MySQL 8 (mysql2)
Portal security	JWT tokens (jose), bcryptjs password hashing, zod input validation, nodemailer for OTP emails
Portal map and ward lookup	Google Maps with a Leaflet fallback; ward found from the map pin using the ward boundary file
Portal department routing	Keyword rules, optional gpt-4o-mini
Grievance export and synthetic history	Node.js scripts (severity, duplicate and language rules in intel.js); Excel export via openpyxl
Police dataset	TypeScript run with tsx, CSV output (mysql2 optional)
PWD dataset	Python standard library only: seeded random numbers, lake water-balance simulation
Hospital dataset	Python and pandas, seeded per date and hospital

3. District intelligence layer (district_intel/, built)

Function	Tech stack
Loading and normalising the 8 sources	Python 3.11, pandas, NumPy

Function	Tech stack
Reference lists: taluks, departments, 32 categories, statuses, facilities	YAML files
Ward, zone and taluk placement, ward adjacency, zone outlines	matplotlib.path (point-in-polygon), SciPy cKDTree (nearest-ward snapping); no PostGIS needed
Text features: language, Tanglish, hazards, vulnerable places, casualties	Python regex (ported from the portal) and SimHash fingerprints
Category classification (English, Tamil, Tanglish)	scikit-learn: character n-gram TF-IDF plus logistic regression
News incident filter	Logistic regression on character n-grams plus multilingual embeddings; trained on rule-based and 300 hand labels, scored with 5-fold cross-validation
Multilingual embeddings	sentence-transformers on PyTorch (CPU), model intfloat/multilingual-e5-small, vectors cached on disk
News story grouping	TF-IDF similarity plus embeddings, then union-find
Duplicates within a source	KD-tree candidate search, rules, union-find
Same incident across sources	KD-tree candidate search, text similarity, logistic-regression scorer with a learned threshold, union-find
Severity, priority, deadlines	Rule tables in Python; priority recomputed at build time
Shared rain calendar for the generators	Python, passed through the generators' own --rain-days and --rainfall-csv options
Scenario overlay	NumPy seeded random numbers, SciPy Poisson
Spike alerts	SciPy Poisson probabilities against pandas rolling baselines
Hotspots	scikit-learn DBSCAN (distance on the globe), Getis-Ord Gi* computed in NumPy
Hospital, lake and air-quality trends	pandas moving averages and z-scores, NumPy trend slope
KPIs and daily counts	pandas
Agents: Data Steward, Action Planner, Gap Finder, Linker, Watchdog, Briefing	Plain Python workflows; a regex checks every number in briefings; optional OpenAI client (off, no key set)
Data-quality checks and source health	pandas, plus each collector's own run-log files
Storage	SQLite (Python built-in sqlite3; 32 tables, 6 views), CSV copies, JSON and GeoJSON feeds for the dashboard
Refresh and scheduling	Each collector's own command line via subprocess; Windows Task Scheduler (DistrictIntel, every 30 minutes) running a .bat
Tests, reports, version control	pytest (18 tests), Markdown reports, git

4. Dashboard

Function	Status	Tech stack
Current prototype (the artifact)	Built	One HTML file, plain JavaScript, hand-drawn SVG map and charts, Google Fonts; data generated in the browser
Real Collector dashboard	Next step	Recommended: the portal's /collector page (Next.js 14, already in place), reading district_intel.db or the JSON feeds; Leaflet with the ward and zone GeoJSON; Recharts or ECharts; TanStack Table; the portal's existing login and roles

Deliberately not in the stack

- **No PostgreSQL/PostGIS or vector database.** SQLite, SciPy and cached NumPy vectors cover district scale with no server to install. Postgres with PostGIS and pgvector is the upgrade path if this grows to many districts.
- **No GPU and no paid API by default.** Everything runs on a laptop CPU. The LLM steps are optional and off.
- **No Claude models wired in yet.** The code only has an OpenAI client, because the repo already used gpt-4o-mini. Claude Haiku 4.5 or Sonnet 5 would sit behind the same optional switch.